



A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA

UNIVERSITÀ DEGLI STUDI DI PERUGIA
DIPARTIMENTO DI INGEGNERIA

Tesina Finale di
Algoritmi e Strutture Dati

Corso di Laurea in Ingegneria Informatica ed Elettronica — A.A. 2025–2026

**Titolo
molto
lungo**

Docente
Prof. Emilio Di Giacomo

Studente
Romeo Miscio
Matricola: 364672

Ultimo aggiornamento: 16 giugno 2026

Indice

1	Il problema della generazione di labirinti	1
2	Modellazione formale tramite Teoria dei Grafi	1
3	Obiettivi del progetto	2
4	Algoritmi implementati	2
4.1	Randomized Depth-First Search (Recursive Backtracker)	2
4.1.1	Scelte implementative e gestione dello Stack	2
4.1.2	Analisi della Complessità Teorica	3
4.2	Algoritmo di Kruskal Randomizzato	3
4.2.1	La struttura dati Disjoint-Set (Union-Find)	3
4.2.2	Analisi della Complessità Teorica	4
4.3	Algoritmo di Prim Modificato	4
4.3.1	Scelte implementative sulla Frontiera	4
4.3.2	Analisi della Complessità Teorica	4

1 Il problema della generazione di labirinti

Il problema della generazione di labirinti perfetti consiste nel ricavare una struttura planare complessa dotata di un'unica interconnessione spaziale priva di interruzioni (regioni isolate) e priva di percorsi alternativi chiusi (cicli) [1].

In ambito informatico e computazionale, un labirinto non rappresenta semplicemente un elemento ludico, ma si configura come un eccellente banco di prova per lo studio della topologia strutturale, della teoria dei grafi e dell'efficienza algoritmica nell'esplorazione spaziale. Trova inoltre ampie applicazioni nella generazione procedurale di contenuti (PCG) nei videogiochi, nella pianificazione di percorsi per la robotica mobile e nella simulazione di reti di micro-canali biologici o idraulici.

2 Modellazione formale tramite Teoria dei Grafi

Il modo formalmente più rigoroso per descrivere la topologia di un labirinto adotta le definizioni della teoria dei grafi. Una griglia bidimensionale predeterminata (sia essa rettangolare, esagonale o circolare) può essere mappata direttamente su un grafo non diretto e pesato $G = (V, E)$, in cui:

- L'insieme dei vertici V rappresenta l'insieme delle celle discrete che compongono lo spazio calpestabile del labirinto, con $|V| = n$.
- L'insieme degli archi E rappresenta l'insieme di tutti i passaggi originariamente potenziali (ovvero l'assenza di un muro divisorio) tra celle geometricamente adiacenti.

Generare un labirinto strutturalmente valido significa individuare un sottografo accatstato $G' = (V, E')$ tale che $G' \subseteq G$, che rispetti due vincoli stringenti:

1. **Connettività:** Il sottografo G' deve essere strettamente connesso. Se G' non fosse connesso, esisterebbero regioni isolate e strutturalmente irraggiungibili all'interno della griglia, invalidando l'integrità del labirinto.
2. **Alinearità (Assenza di cicli):** Il sottografo G' deve essere aciclico. La presenza di un ciclo implicherebbe l'esistenza di cammini multipli per connettere due nodi qualsiasi, riducendo la complessità intrinseca del labirinto e violando la definizione classica di "labirinto perfetto".

Dalle proprietà fondamentali della teoria dei grafi, un sottografo di G che contiene tutti i vertici V , è connesso ed è privo di cicli è per definizione un **Albero di Copertura** (o *Spanning Tree*). Di conseguenza, l'intero problema della generazione procedurale di labirinti si riduce all'estrazione di uno **Spanning Tree casuale** partendo dal grafo iniziale della griglia. I muri del labirinto finito saranno costituiti esattamente dal complemento degli archi selezionati, ovvero dall'insieme degli archi esclusi $E \setminus E'$.

3 Obiettivi del progetto

L'obiettivo principale di questo lavoro di tesi è implementare, analizzare e confrontare sperimentalmente le prestazioni e le caratteristiche topologiche delle principali famiglie di algoritmi per l'estrazione di Spanning Tree casuali presenti in letteratura.

Nello specifico, la suite software sviluppata in linguaggio Java mira a ricoprire l'intero spettro algoritmico descritto nei requisiti ministeriali, dividendoli per paradigmi di progettazione:

- **Algoritmi basati su Tree/Graph Traversal:** Randomized Depth-First Search (Recursive Backtracker) e Randomized Breadth-First Search.
- **Algoritmi basati su Minimum Spanning Tree adattati:** Algoritmo di Kruskal randomizzato e Algoritmo di Prim modificato.
- **Algoritmi basati su Random Walk:** Algoritmo di Aldous-Broder e Algoritmo di Wilson.
- **Algoritmi geometrici o a divisione spaziale:** Algoritmo di Divisione Ricorsiva ed Eller's Algorithm.

Oltre all'analisi prestazionale pura asintotica e di wall-clock, il progetto si propone di integrare un motore grafico per la visualizzazione dinamica passo-passo della crescita dell'albero, controllato da un'euristica di temporizzazione per garantire la fluidità dell'animazione a prescindere dal dominio di calcolo impostato.

4 Algoritmi implementati

In questo capitolo vengono analizzati in dettaglio i primi tre algoritmi sviluppati per la generazione di labirinti perfetti. Ciascun approccio interpreta il problema dell'estrazione di uno Spanning Tree casuale sfruttando strutture dati e paradigmi logici differenti.

4.1 Randomized Depth-First Search (Recursive Backtracker)

L'algoritmo basato sulla ricerca in profondità (DFS) modificata è un approccio classico di tipo *backtracking*. Esso tende a generare labirinti caratterizzati da un basso numero di diramazioni corte e da corridoi marcatamente lunghi e tortuosi (frequentemente vicoli ciechi profondi), proprietà topologica nota in letteratura come "alto fattore di fiumi".

4.1.1 Scelte implementative e gestione dello Stack

La formulazione puramente ricorsiva dell'algoritmo DFS soffre intrinsecamente del limite di allocazione dello stack di memoria della Java Virtual Machine (*StackOverflowError*) quando la dimensione della griglia $n = |V|$ scala linearmente. Per ovviare a questa limitazione hardware, si è optato per una **vettorializzazione esplicita dello stack** mediante la struttura dati `java.util.Stack`.

Il comportamento del singolo passo operativo, fondamentale per agganciare il ciclo di animazione della veste grafica senza bloccare il thread di rendering, è regolato dalla seguente logica a stati stabili:

1. Viene esaminata la cella in cima allo stack senza estrarla (`peek()`).
2. Si interroga l'intorno geometrico della cella per estrarre l'insieme dei vicini non ancora visitati.
3. Se l'insieme è non vuoto, viene selezionato casualmente un vicino, viene rimosso il muro divisorio tra le due celle, la nuova cella viene marcata come visitata e inserita nello stack (`push()`).
4. Se non vi sono vicini validi (vicolo cieco), si effettua il ritorno a ritroso rimuovendo la cella corrente dallo stack (`pop()`).

4.1.2 Analisi della Complessità Teorica

In una griglia regolare planare, il grado massimo di ogni vertice è limitato superiormente da una costante ($\deg(v) \leq 4$). Di conseguenza, il numero totale di archi è lineare rispetto al numero dei nodi, ovvero $|E| = \mathcal{O}(V)$. Poiché ogni cella viene inserita ed estratta dallo stack esattamente una volta, il ciclo interno esegue al più $2|V|$ iterazioni. L'accesso e la modifica delle proprietà della cella (muri e flag di visita) avvengono in tempo costante $\mathcal{O}(1)$. La complessità temporale asintotica dell'algoritmo è pertanto strettamente lineare:

$$\mathcal{O}(V)$$

Lo spazio aggiuntivo occupato dallo stack nel caso pessimo (un unico corridoio lineare senza diramazioni) coincide con l'altezza massima dell'albero, richiedendo una complessità spaziale pari a $\mathcal{O}(V)$.

4.2 Algoritmo di Kruskal Randomizzato

L'adattamento dell'algoritmo di Kruskal per la generazione di labirinti opera trattando tutti i muri divisorii della griglia come un insieme di archi non pesati. Al posto di minimizzare il costo totale ordinando gli archi, l'algoritmo estrae un arco in modo puramente casuale a ogni iterazione, fondendo progressivamente i sotto-alberi isolati.

4.2.1 La struttura dati Disjoint-Set (Union-Find)

Per controllare in tempo quasi-costante se l'abbattimento di un muro introduce un ciclo indesiderato, è stata implementata da zero una foresta di insiemi disgiunti (**Disjoint-Set**) dotata di due ottimizzazioni fondamentali:

- **Unione per Rango (Union by Rank):** Attacca sempre l'albero con altezza minore sotto la radice dell'albero con altezza maggiore, limitando la crescita logaritmica delle strutture.
- **Compressione dei Cammini (Path Compression):** Durante la fase di ricerca (`find`), ogni nodo visitato viene agganciato direttamente alla radice dell'insieme, appiattendolo la struttura per le invocazioni successive.

4.2.2 Analisi della Complessità Teorica

Inizialmente la lista contiene la totalità degli archi possibili della griglia, pari a $|E| \approx 2|V|$. Il mescolamento iniziale (`Collections.shuffle`) richiede tempo lineare $\mathcal{O}(E)$. A ogni passo estrattivo, l'algoritmo esegue due operazioni di `find` e al più una operazione di `union`. Sotto le ipotesi di compressione dei cammini e unione per rango, la complessità ammortizzata di ciascuna operazione è limitata superiormente dalla funzione inversa di una crescita esponenziale raddoppiata, espressa mediante la funzione di iterazione di Ackermann $\alpha(n)$. La complessità temporale complessiva per elaborare tutti gli archi è:

$$\mathcal{O}(E \cdot \alpha(V))$$

Poiché nella nostra griglia $|E| = \mathcal{O}(V)$ e la funzione α assume un valore pratico minore di 5 per qualsiasi input realistico, l'algoritmo opera in tempo di *wall-clock* quasi-lineare. Lo spazio aggiuntivo per mantenere i vettori dei padri e dei ranghi è pari a $\mathcal{O}(V)$.

4.3 Algoritmo di Prim Modificato

L'algoritmo di Prim si sviluppa espandendo radialmente un unico sotto-albero a partire da una cella di innesco casuale. Mantiene costantemente traccia di una struttura di “frontiera”, definita come l'insieme di tutti i muri che separano una cella già inclusa nel labirinto da una cella non ancora esplorata.

4.3.1 Scelte implementative sulla Frontiera

Nell'implementazione didattica standard dell'algoritmo di Prim (utilizzato per la ricerca del Minimum Spanning Tree classico), la frontiera viene memorizzata in una coda a priorità (*Min-Heap*) basata sul peso degli archi. Nel contesto della generazione procedurale casuale, l'assenza di pesi deterministici ci permette di sostituire lo heap con una lista dinamica indicizzata (`java.util.ArrayList`).

La rimozione di un elemento casuale da una lista basata su array richiede lo spostamento degli elementi successivi, operazione intrinsecamente inefficiente se eseguita al centro della struttura ($\mathcal{O}(N)$). Per mantenere il costo microscopico nell'interfaccia a passi, si memorizzano i riferimenti strutturali espliciti dei muri (`PrimWall`) e si effettua un campionamento ad accesso diretto.

4.3.2 Analisi della Complessità Teorica

Ogni cella viene visitata una sola volta e aggiunge al più 4 muri alla lista di frontiera. Di conseguenza, il numero massimo di elementi immessi nella lista è proporzionale a $4|V|$. L'estrazione casuale di un indice da una struttura `ArrayList` avviene in tempo costante $\mathcal{O}(1)$, ma la rimozione di un elemento richiede nel caso pessimo uno slittamento lineare di memoria pari alla dimensione istantanea della frontiera stessa, che nel regime denso di esplorazione può contenere fino a $\mathcal{O}(V)$ elementi. La complessità temporale complessiva su griglia si attesta quindi a:

$$\mathcal{O}(V^2)$$

Dal punto di vista spaziale, la frontiera immagazzina al più il perimetro dinamico di crescita dell'albero, occupando in memoria uno spazio delimitato superiormente da $\mathcal{O}(V)$.

Riferimenti bibliografici

- [1] Wikipedia contributors, “Maze generation algorithm,” *Wikipedia, The Free Encyclopedia*, https://en.wikipedia.org/wiki/Maze_generation_algorithm.